

Проект: batych_RIP
Путь: /home/yashnyk/ollama/batych_RIP
Файлов: 14

=====

FILE: config.yaml

=====

```
llm_model: "llama3"
vision_model: "llava"
temperature: 0.1
input_main: "data/input/main"
input_control: "data/input/control"
input_examples: "data/input/examples"
output_path: "data/output"
human_mode: false
max_iterations: 10
```

=====

FILE: graph/nodes.py

=====

```
import logging
from graph.state import GraphState
from llm.client import generate_text
from utils.docx_builder import save_section_to_docx
from utils.drawio_renderer import render_drawio_to_png
def init_project(state: GraphState):
    logging.info("Инициализация проекта. Ожидание ввода пользователя.")
    title = input("Введите название работы: ")
    problem = input("Введите проблему: ")
    goal = input("Введите цель работы: ")
    tasks = input("Введите задания на ВКР: ")
    return {"thesis_title": title, "problem_statement": problem, "thesis_goal": goal, "thesis_
tasks": tasks}
def drafter(state: GraphState):
    current_iter = state.get("current_iteration", 0) + 1
    logging.info(f"Агент-Писатель начал работу (Итерация {current_iter} из {state.get('max_ite
rations')}).")
    with open("prompts/drafter_prompt.txt", "r", encoding="utf-8") as f:
        system_prompt = f.read()
        prompt = f"{system_prompt}\nТема: {state.get('thesis_title', '')}\nРаздел: {state.get('cur
rent_section', '')}\nПравила: {state.get('pdf_context', '')}"
        if state.get("user_decision"):
            prompt += f"\nУказание: {state['user_decision']}"
        draft = generate_text(prompt, model_name="llama3")
        logging.info("Агент-Писатель успешно сгенерировал текст.")
        return {"draft_text": draft, "user_decision": "", "current_iteration": current_iter}
def reviewer(state: GraphState):
    logging.info("Агент-Редактор проверяет черновик на соответствие правилам.")
    with open("prompts/reviewer_prompt.txt", "r", encoding="utf-8") as f:
        system_prompt = f.read()
        prompt = f"{system_prompt}\nПравила: {state.get('pdf_context', '')}\nЧерновик: {state.get(
'draft_text', '')}"
        review_result = generate_text(prompt, model_name="llama3")
        errors = []
        if "ОДОБРЕНО" not in review_result:
            logging.warning("Агент-Редактор нашел ошибки в тексте.")
            errors.append(review_result)
        else:
            logging.info("Текст ОДОБРЕНО Агентом-Редактором.")
        return {"errors": errors}
def error_gateway(state: GraphState):
    if state.get("human_mode"):
        logging.warning("Запуск шлюза. Требуется вмешательство человека.")
        print("Агент-Редактор нашел разночтения или предлагает варианты:")
        for error in state.get("errors", []):
            print(error)
```

```

        print("\n--- Текст черновика ---")
        print(state.get("draft_text", ""))
        print("-----")
        decision = input("Введите правки для Писателя (или напишите 'ПРИНЯТЬ'): ")
        return {"user_decision": decision, "errors": []}
    else:
        logging.warning("Автономный режим. Формирование задания на исправление.")
        decision = "Исправь текст с учетом этих замечаний от редактора:\n" + "\n".join(state.get("errors", []))
        return {"user_decision": decision, "errors": []}
def diagram_maker(state: GraphState):
    logging.info("Агент-Архитектор создает схему (XML для draw.io).")
    with open("prompts/diagram_prompt.txt", "r", encoding="utf-8") as f:
        system_prompt = f.read()
    prompt = f"{system_prompt}\nСделай схему для текста: {state.get('draft_text', '')}"
    xml_code = generate_text(prompt, model_name="llama3")
    start = xml_code.find("<mxGraphModel>")
    end = xml_code.find("</mxGraphModel>")
    if start != -1 and end != -1:
        xml_code = xml_code[start:end+15]
    logging.info("Схема успешно сгенерирована.")
    return {"diagram_xml": xml_code}
def doc_writer(state: GraphState):
    logging.info("Сохранение результатов в Word-документ.")
    image_path = None
    if state.get("diagram_xml") and "<mxGraphModel>" in state.get("diagram_xml"):
        try:
            logging.info("Рендеринг XML в PNG изображение.")
            image_path = render_drawio_to_png(state["diagram_xml"])
        except Exception as e:
            logging.error(f"Ошибка рендеринга картинки: {e}")
            image_path = None
    save_section_to_docx(state.get("current_section", "Раздел"), state.get("draft_text", ""),
    image_path)
    logging.info("Раздел успешно сохранен в документ.")
    return {"diagram_xml": "", "draft_text": ""}

```

```

=====
FILE: graph/state.py
=====

```

```

from typing import TypedDict, List
class GraphState(TypedDict):
    thesis_title: str
    problem_statement: str
    thesis_goal: str
    thesis_tasks: str
    current_section: str
    draft_text: str
    pdf_context: str
    errors: List[str]
    user_decision: str
    diagram_xml: str
    human_mode: bool
    max_iterations: int
    current_iteration: int

```

```

=====
FILE: graph/workflow.py
=====

```

```

from langgraph.graph import StateGraph, END
from graph.state import GraphState
from graph.nodes import init_project, drafter, reviewer, error_gateway, diagram_maker, doc_writer
def router(state: GraphState):
    if len(state.get("errors", [])) > 0:

```

```

        if state.get("current_iteration", 0) >= state.get("max_iterations", 3):
            if "cxema" in state.get("draft_text", "").lower():
                return "diagram_maker"
            return "doc_writer"
        return "error_gateway"
    if "cxema" in state.get("draft_text", "").lower():
        return "diagram_maker"
    return "doc_writer"
def gateway_router(state: GraphState):
    decision = state.get("user_decision", "").strip().lower()
    if decision in ["принять", "ok", "ок", "далее"]:
        if "cxema" in state.get("draft_text", "").lower():
            return "diagram_maker"
        return "doc_writer"
    return "drafter"
workflow = StateGraph(GraphState)
workflow.add_node("init_project", init_project)
workflow.add_node("drafter", drafter)
workflow.add_node("reviewer", reviewer)
workflow.add_node("error_gateway", error_gateway)
workflow.add_node("diagram_maker", diagram_maker)
workflow.add_node("doc_writer", doc_writer)
workflow.set_entry_point("init_project")
workflow.add_edge("init_project", "drafter")
workflow.add_edge("drafter", "reviewer")
workflow.add_conditional_edges("reviewer", router)
workflow.add_conditional_edges("error_gateway", gateway_router)
workflow.add_edge("diagram_maker", "doc_writer")
workflow.add_edge("doc_writer", END)
app = workflow.compile()

```

```

=====
FILE: llm/client.py
=====

```

```

import requests
import json
import base64
def generate_text(prompt, model_name="qwen2.5:14b"):
    url = "http://localhost:11434/api/generate"
    data = {
        "model": model_name,
        "prompt": prompt,
        "stream": False
    }
    headers = {"Content-Type": "application/json"}
    response = requests.post(url, data=json.dumps(data), headers=headers)
    if response.status_code == 200:
        result = response.json()
        return result["response"]
    return "Ошибка соединения"
def generate_from_image(prompt, image_path, model_name="llava"):
    with open(image_path, "rb") as image_file:
        img_base64 = base64.b64encode(image_file.read()).decode("utf-8")
    url = "http://localhost:11434/api/generate"
    data = {
        "model": model_name,
        "prompt": prompt,
        "stream": False,
        "images": [img_base64]
    }
    headers = {"Content-Type": "application/json"}
    response = requests.post(url, data=json.dumps(data), headers=headers)
    if response.status_code == 200:
        result = response.json()
        return result["response"]
    return "Ошибка распознавания картинки"

```

```
=====
FILE: main.py
=====
```

```
import yaml
import os
import logging
import shutil
from graph.workflow import app
from utils.pdf_parser import extract_data_from_pdf
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(module)s: %(message)s")
def load_config():
    with open("config.yaml", "r", encoding="utf-8") as f:
        return yaml.safe_load(f)
def main():
    config = load_config()
    pdf_context_text = ""
    control_dir = config.get("input_control", "data/input/control")
    examples_dir = config.get("input_examples", "data/input/examples")
    main_dir = config.get("input_main", "data/input/main")
    output_dir = config.get("output_path", "data/output")
    human_mode = config.get("human_mode", False)
    max_iterations = config.get("max_iterations", 3)
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
    base_docx = os.path.join(output_dir, "thesis.docx")
    if not os.path.exists(base_docx) and os.path.exists(main_dir):
        for filename in os.listdir(main_dir):
            if filename.endswith(".docx"):
                shutil.copy(os.path.join(main_dir, filename), base_docx)
                logging.info(f"Взял базовый шаблон документа: {filename}")
                break
    if os.path.exists(control_dir):
        for filename in os.listdir(control_dir):
            if filename.endswith(".pdf"):
                file_path = os.path.join(control_dir, filename)
                logging.info(f"Чтение нормативов: {filename}")
                pdf_context_text += "\n[НОРМАТИВЫ]:\n" + extract_data_from_pdf(file_path)
    if os.path.exists(examples_dir):
        for filename in os.listdir(examples_dir):
            if filename.endswith(".pdf"):
                file_path = os.path.join(examples_dir, filename)
                logging.info(f"Чтение примеров: {filename}")
                pdf_context_text += "\n[ПРИМЕРЫ ПРОШЛЫХ ЛЕТ]:\n" + extract_data_from_pdf(file_
path)
    initial_state = {
        "thesis_title": "",
        "problem_statement": "",
        "thesis_goal": "",
        "thesis_tasks": "",
        "current_section": "Введение",
        "draft_text": "",
        "pdf_context": pdf_context_text,
        "errors": [],
        "user_decision": "",
        "diagram_xml": "",
        "human_mode": human_mode,
        "max_iterations": max_iterations,
        "current_iteration": 0
    }
    logging.info("Запуск графа генерации ВКР...")
    for output in app.stream(initial_state, config={"configurable": {"thread_id": "1"}}):
        for key, value in output.items():
            logging.info(f"--- Завершен узел: {key} ---")
if __name__ == "__main__":
    main()
```

```
=====
FILE: project_to_pdf.py
=====
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from pathlib import Path
import os

from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.pdfgen import canvas
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont

ROOT = Path(".").resolve()
OUTPUT = ROOT / "project_dump.pdf"

INCLUDE_EXTS = {
    ".py", ".txt", ".md", ".json", ".yaml", ".yml",
    ".toml", ".ini", ".cfg", ".sh"
}
EXCLUDE_DIRS = {
    ".git", ".venv", "venv", "__pycache__", ".idea",
    ".vscode", "node_modules", "dist", "build"
}
EXCLUDE_FILES = {OUTPUT.name}

PAGE_W, PAGE_H = A4
LEFT = 15 * mm
RIGHT = 15 * mm
TOP = 15 * mm
BOTTOM = 15 * mm

FONT_PATHS = [
    "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
    "/usr/share/fonts/truetype/liberation2/LiberationMono-Regular.ttf",
]
FONT_NAME = "DejaVuMono"
FONT_SIZE = 9
LINE_STEP = 11 # points

def register_font():
    for path in FONT_PATHS:
        if os.path.exists(path):
            pdfmetrics.registerFont(TTFont(FONT_NAME, path))
            return
    raise RuntimeError("Не найден TTF-шрифт с поддержкой кириллицы")

def should_skip(path: Path) -> bool:
    if path.name in EXCLUDE_FILES:
        return True
    for part in path.parts:
        if part in EXCLUDE_DIRS:
            return True
    return False

def iter_files(root: Path):
    for p in sorted(root.rglob("*")):
        if p.is_file() and not should_skip(p):
            if p.suffix.lower() in INCLUDE_EXTS or p.name == ".gitignore":
                yield p
```

```

def read_text(path: Path) -> str:
    for enc in ("utf-8", "utf-8-sig", "cp1251", "latin-1"):
        try:
            return path.read_text(encoding=enc)
        except UnicodeDecodeError:
            continue
    except Exception as e:
        return f"<ERROR READING FILE: {e}>"
    return "<BINARY OR UNREADABLE CONTENT>"

def wrap_text_line(line: str, max_width: float):
    line = line.replace("\t", "    ")
    if line == "":
        return [""]

    parts = []
    current = ""

    for ch in line:
        test = current + ch
        width = pdfmetrics.stringWidth(test, FONT_NAME, FONT_SIZE)
        if width <= max_width:
            current = test
        else:
            if current:
                parts.append(current)
                current = ch

    if current:
        parts.append(current)

    return parts if parts else [""]

class PdfWriter:
    def __init__(self, out_path: Path):
        self.c = canvas.Canvas(str(out_path), pagesize=A4)
        self.page_num = 1
        self.text = None
        self.start_page()

    def start_page(self):
        self.c.setFont(FONT_NAME, FONT_SIZE)
        self.text = self.c.beginText()
        self.text.setTextOrigin(LEFT, PAGE_H - TOP)
        self.text.setFont(FONT_NAME, FONT_SIZE)
        self.text.setLeading(LINE_STEP)

    def finish_page(self):
        self.c.drawText(self.text)
        self.c.setFont(FONT_NAME, 8)
        self.c.drawRightString(PAGE_W - RIGHT, 8 * mm, f"Стр. {self.page_num}")
        self.c.showPage()
        self.page_num += 1

    def ensure_space(self):
        if self.text.getY() <= BOTTOM:
            self.finish_page()
            self.start_page()

    def write_line(self, line=""):
        self.ensure_space()
        self.text.textLine(line)

    def write_block(self, lines):
        for line in lines:
            self.write_line(line)

```

```

def save(self):
    self.c.drawText(self.text)
    self.c.setFont(FONT_NAME, 8)
    self.c.drawRightString(PAGE_W - RIGHT, 8 * mm, f"Стр. {self.page_num}")
    self.c.save()

def main():
    register_font()
    writer = PdfWriter(OUTPUT)

    max_width = PAGE_W - LEFT - RIGHT
    files = list(iter_files(ROOT))

    writer.write_line(f"Проект: {ROOT.name}")
    writer.write_line(f"Путь: {ROOT}")
    writer.write_line(f"Файлов: {len(files)}")
    writer.write_line("")

    for path in files:
        rel = path.relative_to(ROOT)
        content = read_text(path)

        writer.write_line("=" * 80)
        for part in wrap_text_line(f"FILE: {rel}", max_width):
            writer.write_line(part)
        writer.write_line("=" * 80)
        writer.write_line("")

        for raw in content.splitlines():
            wrapped = wrap_text_line(raw, max_width)
            for part in wrapped:
                writer.write_line(part)

        writer.write_line("")
        writer.write_line("")

    writer.save()
    print(f"Сохранено: {OUTPUT}")

if __name__ == "__main__":
    main()

```

```

=====
FILE: prompts/diagram_prompt.txt
=====

```

Ты системный архитектор. По переданному описанию создай схему. Выведи только валидный XML код для draw.io (тег <mxGraphModel> и все его содержимое). Не пиши никакой дополнительный текст.

```

=====
FILE: prompts/drafter_prompt.txt
=====

```

Ты технический писатель. Твоя задача — написать раздел технической документации для выпускной квалификационной работы.

Строгие правила:

1. Запрещено использовать сложные, перегруженные слова (например, "аккумулируются", "синергия" и т.д.). Пиши простым, понятным техническим языком.
 2. Строгий запрет на упоминание экономики, бюджетов, смет и экономической выгоды. Работа только о технической (CAN, LIN, MQTT).
 3. Если ты пишешь программный код, он должен выглядеть как код любителя: с нормальными отступами, но СТРОГО БЕЗ пустых строк и БЕЗ комментариев внутри кода.
 4. Весь сгенерированный текст должен быть СТРОГО на русском языке.
- Напиши черновик текста на основе переданных данных.

```
=====
FILE: prompts/reviewer_prompt.txt
=====
```

Ты технический нормоконтролер и редактор. Твоя задача – сверить черновик текста с загруженными правилами из нормативных документов.

1. Проверь, нет ли фактических ошибок в описании стандартов связи.
2. Проверь, не нарушено ли правило простых слов.

Если есть ошибки или ты видишь несколько вариантов развития архитектуры сети, напиши список замечаний.

Если текст полностью соответствует правилам и не требует правок, выведи ровно одно слово: ОДОбРЕНО.

```
=====
FILE: requirements.txt
=====
```

```
=====
FILE: utils/docx_builder.py
=====
```

```
import os
from docx import Document
from docx.shared import Pt
def save_section_to_docx(section_title, text, image_path=None, file_path="data/output/thesis.docx"):
    if os.path.exists(file_path):
        doc = Document(file_path)
    else:
        doc = Document()
    doc.add_heading(section_title, level=1)
    paragraph = doc.add_paragraph(text)
    for run in paragraph.runs:
        run.font.name = 'Times New Roman'
        run.font.size = Pt(14)
    if image_path and os.path.exists(image_path):
        doc.add_picture(image_path)
    doc.save(file_path)
    return file_path
```

```
=====
FILE: utils/drawio_renderer.py
=====
```

```
import subprocess
import os
import uuid
def render_drawio_to_png(xml_content, output_dir="data/output/images"):
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
    file_id = str(uuid.uuid4())
    drawio_path = os.path.join(output_dir, f"{file_id}.drawio")
    png_path = os.path.join(output_dir, f"{file_id}.png")
    with open(drawio_path, "w", encoding="utf-8") as f:
        f.write(xml_content)
    subprocess.run(["drawio", "-x", "-f", "png", "-o", png_path, drawio_path], check=True)
    return png_path
```

```
=====
FILE: utils/pdf_parser.py
=====
```



```

import pymupdf
import os
def extract_data_from_pdf(file_path, output_image_dir="data/output/images"):
    if not os.path.exists(output_image_dir):
        os.makedirs(output_image_dir)
    document = pymupdf.open(file_path)
    full_text = ""
    for page_num in range(len(document)):
        page = document.load_page(page_num)
        full_text += page.get_text()
        image_list = page.get_images()
        for img_index, img in enumerate(image_list):
            xref = img[0]
            base_image = document.extract_image(xref)
            image_bytes = base_image["image"]
            image_ext = base_image["ext"]
            image_path = os.path.join(output_image_dir, f"page_{page_num}_img_{img_index}.{image_ext}")
            with open(image_path, "wb") as img_file:
                img_file.write(image_bytes)
    return full_text

```